PROTECTION.md

Project: Arati Business ERP
Document Version: 1.0
Status: Security & Protection Baseline — Draft for Production Design
Primary Scope: Distribution + Wholesale + Retail + Inventory ERP
Accounting Integration: TallyPrime

Related Documents

- BRD.md
- SRS_v2.0.md
- ARCHITECTURE_v2.0.md
- TECHSTACK_v1.1.md
- DATABASE_v1.0.md
- BUSINESS-RULES_v1.0.md
- API.md
- TALLY-INTEGRATION.md

---

1. Purpose

This document defines the security, protection, integrity, and recovery model for the ERP.

It protects against:

1. unauthorized users
2. unauthorized devices and sessions
3. unauthorized network access
4. malicious or compromised clients
5. stolen sessions/tokens
6. unauthorized API calls
7. privilege escalation
8. accidental or malicious data changes
9. inventory manipulation
10. financial manipulation
11. Tally integration abuse
12. unsafe file uploads
13. data leakage
14. database compromise
15. supply-chain and dependency attacks
16. destructive attacks and ransomware

17. operational failures and recovery scenarios

Security is not a login feature. The ERP must use **defense in depth**.

Internet / Network
↓
Edge Protection
↓
HTTPS / TLS
↓
Application Access
↓
Authentication
↓
Authorization
↓
Input Validation
↓
Business Rules
↓
Transaction Boundary
↓
Database Constraints
↓
Audit / Monitoring
↓
Backup / Recovery

No single layer is the only protection.

---

2. Security Principles

2.1 Least Privilege

Every user, service, API credential, database account, worker, and integration receives only the permissions required for its job.

2.2 Deny by Default

If access has not been explicitly granted, it is denied.

2.3 Never Trust the Client

The browser is untrusted.

A client may alter:

- prices
- quantities
- IDs
- roles
- branch IDs
- warehouse IDs
- document statuses
- totals
- tax values
- discount values
- payment allocations
- stock quantities

All security-sensitive and business-critical decisions must be enforced server-side.

2.4 Defense in Depth

A failure in one control must not automatically compromise the whole system.

2.5 Secure by Default

New users, APIs, resources, integrations, files, and deployments should start with the safest reasonable configuration.

2.6 Integrity Before Convenience

For stock and financial operations, preventing an incorrect transaction is more important than silently allowing it.

2.7 Complete Auditability

Important actions must be traceable to:

- user or service
- timestamp
- organization
- branch/location
- resource
- resource ID
- request/correlation ID
- action/result
- reason where applicable

2.8 Fail Closed for Security

When authorization or integrity cannot be established safely, the operation should fail rather than guess.

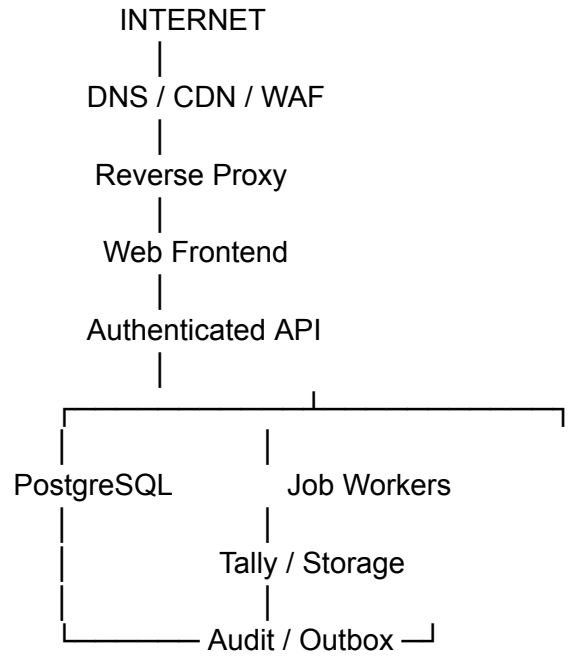
2.9 Recoverability

Security includes the ability to recover from compromise, deletion, corruption, and infrastructure failure.

---

3. Trust Boundaries

The system contains distinct trust zones.



Rules:

- PostgreSQL is never public.
- Internal worker services are never public unless explicitly required.
- Tally is an external trust boundary.
- Object storage is an external trust boundary when hosted outside the application environment.
- The browser is never trusted to enforce permissions.
- Administrative infrastructure is separated from normal user access.

---

4. Outer Network Security

Production infrastructure should expose only necessary services.

Typical public exposure:

Internet
  │
  ├── 443 → HTTPS application entry point
  │
  └── 80 → HTTPS redirect only, if required

The following must not be publicly accessible:

PostgreSQL
Redis
Internal worker ports
Admin databases
Internal service ports
Development tools
Debug interfaces
Management interfaces

Database access is restricted to trusted application/administration networks.

---

5. HTTPS / TLS

Production application traffic must use HTTPS.

Requirements:

- TLS for all external application traffic
- valid certificates
- automatic certificate renewal
- modern TLS configuration
- HTTP-to-HTTPS redirect where appropriate
- no mixed-content resources
- secure cookies
- HSTS where appropriate

Internal encryption should be used where required by the deployment topology.

---

6. Firewall, WAF and Edge Protection

Use infrastructure controls appropriate to the deployment:

- firewall
- reverse proxy
- WAF where internet exposure warrants it
- network segmentation
- DDoS/abuse protection
- restricted administrative access
- monitoring

A WAF is an additional layer. It does not replace secure application code.

---

7. Rate Limiting and Abuse Controls

Rate limiting must be risk-based.

Strict controls should apply to:

- login
- password reset
- session/token refresh
- file uploads
- public APIs
- expensive report/export operations
- authentication-related endpoints
- externally exposed integration endpoints

Application APIs should also enforce:

- request-size limits
- pagination
- expensive-query protection
- export workload limits
- upload limits

Rate limits should be configurable rather than scattered as hard-coded constants.

---

8. Server and Host Hardening

Production hosts should:

- run supported operating systems
- receive security updates
- disable unnecessary services
- restrict administrative access
- prefer SSH keys over passwords where appropriate
- avoid root application processes
- use host firewalls
- synchronize time
- centralize relevant logs
- keep secrets out of shell history

Development machines are not production infrastructure.

---

9. Container Security

If Docker is used:

- use minimal trusted base images
- pin important versions
- scan images for vulnerabilities
- do not run containers as root unless necessary
- minimize Linux capabilities
- use read-only filesystems where practical
- keep secrets out of images
- never bake production `.env` files into images
- rebuild after critical security updates
- use explicit, reproducible build processes

---

10. Database Protection

PostgreSQL is the authoritative transactional database.

It must be protected by:

- private networking
- strong authentication
- encrypted connections where appropriate
- restricted database roles
- least-privilege permissions
- encrypted backups

- controlled migrations
- monitoring
- integrity constraints
- restricted administrative access

The database must never be directly accessible from normal users or browsers.

---

11. Database Account Separation

Where practical, use separate credentials/roles for:

application_runtime
migration_runner
reporting_readonly
backup_operator
administration

The application runtime account must not automatically have unrestricted administrative privileges.

Production migrations require controlled deployment access.

---

12. Secrets Management

Secrets must never be committed to source control.

Examples:

- database passwords
- session secrets
- encryption keys
- Tally credentials
- storage credentials
- SMTP credentials
- API keys
- deployment credentials

Use environment variables or a proper secrets-management system.

Never place secrets in:

source code

frontend JavaScript
Git repositories
screenshots
logs
API responses
seed data
container images
public files

Secret rotation procedures must be documented.

---

13. Authentication

Authentication must establish the identity of the user securely.

Minimum requirements:

- strong password hashing
- login protection
- secure session handling
- expiration
- logout/revocation
- password reset
- account disablement
- failed-login monitoring
- protection against credential stuffing

The preferred browser model is secure server-managed sessions using HTTP-only, secure cookies, subject to final architecture confirmation.

Exact authentication implementation remains:

TBD — Technical Decision Required

---

14. Password Protection

Passwords must never be stored in plaintext.

Use an approved modern adaptive password-hashing algorithm, such as Argon2id where supported by the finalized stack.

Passwords must never be:

- logged
- returned by APIs
- emailed as plaintext
- stored unnecessarily in browser storage
- included in support exports

Password reset must use:

- short-lived mechanisms
- single-use tokens
- generic responses that do not reveal whether an account exists

---

15. Multi-Factor Authentication

MFA should be used for high-risk accounts, especially:

- administrators
- finance users
- users with privileged permissions
- remote administrative access

MFA may become mandatory for privileged accounts.

TBD — Business/Security Decision Required: exact MFA scope and recovery policy.

---

16. Session Security

Sessions must support:

- expiration
- revocation
- secure storage
- protection against session theft
- inactivity timeout
- absolute timeout where appropriate
- logout
- session visibility/revocation for privileged users where practical

For browser authentication:

- HTTP-only cookies preferred
- Secure cookies in production

- appropriate SameSite policy
- avoid long-lived sensitive tokens in local storage

Stolen-session mitigation must not rely only on frontend behavior.

---

17. Authorization Model

Authentication answers:

> Who are you?

Authorization answers:

> What are you allowed to do?

Use permission-based authorization, with roles used as a manageable way to assign permissions.

Example permissions:

sales.invoice.create
sales.invoice.post
sales.invoice.cancel

inventory.view
inventory.adjust
inventory.transfer

payments.create
payments.post

users.manage
permissions.manage

tally.sync
tally.reconcile

Authorization must be enforced on the backend.

UI hiding is not authorization.

---

18. Organization / Branch / Warehouse Isolation

Every protected resource must be checked against the authenticated user's permitted scope.

Changing a request field must never grant access:

organizationId
branchId
warehouseId

The backend must derive or verify scope from authenticated permissions.

Cross-branch operations must be explicit and authorized.

---

19. Object-Level Authorization

Every object access must verify permission for that specific object.

Example:

GET /sales-invoices/{id}

The backend must verify:

1. user is authenticated
2. user has the required permission
3. the invoice belongs to an accessible organization/branch
4. warehouse/location scope where relevant
5. any additional business restrictions

This protects against insecure direct object reference and similar authorization failures.

---

20. Privileged Actions

High-risk actions require distinct permissions.

Examples:

- post invoice
- cancel invoice

- adjust stock
- approve purchase
- modify price
- apply price override
- change credit limit
- post payment
- reverse payment
- reconcile Tally
- modify Tally mappings
- manage users
- manage permissions
- access sensitive exports

View permission does not imply modification or posting permission.

---

21. Financial Protection

Financial integrity is critical.

Protect:

- invoice totals
- taxes
- discounts
- payment amounts
- payment allocations
- credit limits
- refunds/returns
- cancellations
- document numbering

The backend must calculate or verify authoritative values.

Never trust client-supplied values such as:

grandTotal
taxTotal
discountTotal
outstandingAmount
stockAvailable

Historical transaction values must remain stable even when current master data changes.

---

22. Inventory Protection

Inventory is a critical security boundary.

The frontend must never directly modify stock balances.

Stock changes occur only through authorized domain operations such as:

goods receipt
sales posting
purchase return
sales return
stock transfer
stock adjustment

Every stock-changing operation must create traceable inventory movement records.

Direct manual editing of stock balances is prohibited.

---

23. Stock Concurrency Protection

The system must prevent race conditions.

Example:

Available stock = 1

User A → sells 1
User B → sells 1

Both requests must not succeed merely because they read the same previous balance.

Use appropriate combinations of:

- database transactions
- row locking
- atomic updates
- unique constraints
- isolation controls
- serializable/retry strategies where justified

TBD — Technical Design Required: final concurrency strategy per inventory operation.

---

24. Serial and Batch Protection

For serial-tracked products:

- a serial must not be sold twice
- concurrent sale attempts must be prevented
- serial state changes must be atomic
- serial movement history must be traceable

For batch-tracked products:

- batch identity must be preserved
- quantities must remain consistent
- expiry/other batch controls must be applied when configured

The authoritative lifecycle is represented by inventory/serial movement history, not by mutable UI state.

---

25. Posted Document Protection

Posted documents should become effectively immutable.

Typical lifecycle:

DRAFT
↓
POSTED

After posting, normal users must not directly modify:

- quantity
- price
- tax
- customer
- warehouse
- payment
- document date
- serial/batch allocation

Corrections use controlled mechanisms such as:

cancellation
reversal
return
credit/debit correction

adjustment
corrective transaction

Exact correction workflow:

TBD — Business Confirmation Required

---

26. Approval Controls

Approval workflows should protect high-risk operations where required.

Possible examples:

- large purchase orders
- high-value discounts
- stock adjustments
- credit-limit overrides
- invoice cancellation
- manual financial adjustments

Approval thresholds and approver roles:

TBD — Business Confirmation Required

---

27. Price Protection

Prices must not be trusted from the browser.

The backend should:

1. determine the applicable price
2. validate requested price
3. validate discounts
4. verify user permission
5. store the final transaction value
6. audit manual overrides

Price overrides require explicit permission when the business allows them.

---

28. Tax Protection

Tax calculations must be performed or validated server-side.

Client-supplied tax totals are never authoritative.

Tax rules must support effective dating/versioning where required so historical transactions retain their original treatment.

GST/tax behavior:

TBD — Business Confirmation Required

---

29. API Protection

Protected endpoints should follow:

Authentication
↓
Authorization
↓
Input Validation
↓
Business Validation
↓
Transaction Boundary
↓
Database Constraints

Reject or safely handle:

- malformed IDs
- invalid enums
- invalid numbers
- impossible quantities
- unauthorized references
- unexpected fields where appropriate
- oversized requests
- invalid state transitions

Errors must not reveal secrets, internal stack traces, or unnecessary sensitive data.

---

30. Injection Protection

Protect against:

- SQL injection
- command injection
- template injection
- XSS
- path traversal
- unsafe expression evaluation
- other interpreter injection applicable to the stack

Use:

- parameterized queries
- ORM/query-builder protections
- strict validation
- output encoding
- safe filesystem APIs
- safe process execution

Never concatenate raw user input into SQL or shell commands.

---

31. XSS Protection

User-controlled data may appear in:

- customer names
- addresses
- notes
- product descriptions
- delivery instructions
- installation notes
- imported data

Use framework output encoding and carefully reviewed sanitization where HTML is genuinely required.

Do not render arbitrary user HTML by default.

---

32. CSRF Protection

If browser authentication uses cookies, state-changing requests must be protected against CSRF.

Use appropriate combinations of:

- SameSite cookies
- CSRF tokens where required
- Origin/Referer validation where appropriate

Exact implementation depends on final authentication architecture.

---

33. CORS

CORS must be explicitly configured.

Do not use unrestricted authenticated API access such as:

Access-Control-Allow-Origin: *

Trusted application origins should be allow-listed.

Credentials must not be enabled for arbitrary origins.

---

34. File Upload Protection

Attachments are an attack surface.

Controls should include:

- maximum file size
- allowed file types
- file-signature validation where practical
- safe server-generated filenames
- storage outside executable web directories
- malware scanning where practical
- path-traversal protection
- executable-file restrictions
- authorization on downloads
- content-disposition controls where appropriate

Do not trust browser MIME type alone.

Uploaded files should not automatically become publicly accessible.

---

35. Export Protection

Reports and exports may contain large amounts of sensitive business data.

Exports must:

- require authorization
- respect organization/branch scope
- respect role/permission restrictions
- apply reasonable workload limits
- protect generated files
- expire temporary download access
- avoid public URLs
- be audited when sensitive

Large exports may be generated asynchronously.

---

36. Business and Personal Data Protection

The ERP may contain:

- names
- phone numbers
- addresses
- tax identifiers
- payment information
- customer/supplier relationships
- financial records
- operational history

Access must be limited by job responsibility and authorization scope.

Sensitive information should not unnecessarily appear in:

- logs
- error messages
- analytics
- browser telemetry
- notifications
- support dumps

---

37. Logging Protection

Logs are sensitive.

Never log:

- passwords
- authentication tokens
- session secrets
- Tally passwords
- encryption keys
- full payment secrets

Avoid unnecessary personal data.

Where possible, use structured logs with:

- timestamp
- severity
- service/module
- request/correlation ID
- actor/service ID where appropriate
- operation
- result
- safe diagnostic context

---

38. Audit Logging

Important security and business actions must create audit records.

Minimum useful fields:

who
when
what
resource type
resource ID
organization
branch/location
request/correlation ID
result
reason where applicable

For sensitive changes, capture relevant before/after values.

Audit records must:

- be protected from normal users
- not be casually editable
- survive ordinary transaction corrections
- support investigation and reconciliation
- avoid storing unnecessary secrets

Important events include:

- login/security events
- user creation/disablement
- permission changes
- price overrides
- invoice posting/cancellation
- payment posting/reversal
- stock adjustments/transfers
- returns
- credit-limit overrides
- Tally mapping changes
- Tally synchronization failures
- sensitive exports
- administrative actions

---

39. Security Event Monitoring

Monitor for suspicious patterns such as:

- repeated failed logins
- disabled-user login attempts
- unusual login activity
- permission changes
- administrator creation
- unusual stock adjustments
- unusual invoice cancellations
- unusual discounts
- repeated payment failures
- repeated Tally failures
- repeated API errors
- suspicious file uploads
- unexpected administrative activity

Alert thresholds should reflect operational risk.

---

40. Tally Integration Protection

Tally is an external trust boundary.

Requirements:

- credentials protected
- credentials never exposed to browsers
- credentials never returned by ERP APIs
- sync jobs authenticated
- mappings protected
- synchronization auditable
- retries safe
- duplicate voucher creation prevented
- idempotency supported
- reconciliation supported
- sync failures isolated from core ERP transactions

The ERP must continue operating when Tally is unavailable.

Exact Tally communication/authentication method:

TBD — Technical Decision Required

---

41. Background Worker Protection

Workers may process:

Tally synchronization
notifications
report generation
file processing

Workers must operate through controlled internal mechanisms.

Apply:

- least privilege
- authenticated internal execution
- scoped database permissions where practical
- restricted network access
- job validation

- safe retry behavior
- auditability for sensitive jobs

A compromised worker must not automatically imply unrestricted system access.

---

42. Database Integrity Controls

Application logic is not enough.

Where practical, critical rules must also be enforced through PostgreSQL constraints:

- primary keys
- foreign keys
- unique constraints
- check constraints
- non-null constraints
- unique document numbers
- valid relationships
- appropriate exclusion/partial/functional constraints where needed

The database provides an additional protection layer against application defects and race conditions.

---

43. Transaction Integrity

Critical operations must be atomic.

Examples:

- posting a sale
- posting a purchase receipt
- stock transfer
- stock adjustment
- payment posting/allocation
- return posting
- cancellation/reversal

A successful response must not be returned unless the authoritative transaction has committed.

Optional secondary work, such as notifications or external synchronization, must not corrupt or roll back a valid core ERP transaction unless explicitly designed as part of the transaction boundary.

---

44. Idempotency and Replay Protection

Retryable operations must not create duplicate business effects.

Particularly important for:

- payment posting
- invoice posting
- stock operations
- imports
- Tally synchronization
- webhook/integration processing
- background jobs

Use controlled idempotency keys or equivalent operation identifiers where appropriate.

The system should distinguish:

same request retried

from:

new legitimate transaction

---

45. Backup Protection

Backups are part of security.

Backups should be:

- automated
- encrypted
- access-controlled
- separated from the primary system where appropriate
- protected against accidental deletion
- retained according to business requirements
- tested through restoration

A backup that has never been successfully restored is not proven.

Backup credentials must not be shared with normal application users.

---

46. Disaster Recovery

The project must define:

RPO — acceptable data loss
RTO — acceptable recovery time

These are:

TBD — Business/Operations Decision Required

Recovery procedures must cover:

- database failure
- server failure
- corrupted deployment
- accidental deletion
- ransomware/destructive attack
- storage failure
- credential compromise
- Tally outage
- failed deployment/migration

Recovery procedures must be documented and periodically tested.

---

47. Ransomware / Destructive Attack Protection

Important controls include:

- protected or immutable backups where available
- least-privilege service accounts
- restricted administrative access
- endpoint security
- network segmentation
- monitoring
- tested restoration
- controlled deployment credentials

The goal is not merely prevention. The system must be recoverable after compromise.

---

48. Dependency Security

Dependencies are part of the attack surface.

Requirements:

- lock dependency versions
- scan dependencies for known vulnerabilities
- review critical advisories
- remove unused dependencies
- patch security issues promptly
- avoid unmaintained packages for critical functionality
- review new security-sensitive dependencies

Dependency updates must be tested before production deployment.

---

49. Supply-Chain Protection

Protect the source code and build/deployment pipeline.

Recommended controls:

- protected main branch
- pull-request review
- required CI checks
- secret scanning
- dependency scanning
- container scanning
- restricted production deployment
- separate environment credentials
- traceable releases
- controlled build artifacts

Production should not be deployed from an uncontrolled developer workstation.

---

50. Git / Repository Protection

The repository should enforce:

- branch protection
- required reviews
- CI before merge
- secret scanning
- controlled repository access
- auditable history

Never commit:

.env
database passwords
Tally credentials
private keys
API secrets
production database dumps
customer exports
backup archives containing sensitive data

---

51. CI/CD Security

The deployment pipeline should:

1. authenticate securely
2. use short-lived credentials where practical
3. build from trusted source
4. run tests
5. run security checks
6. scan dependencies
7. scan container images where applicable
8. deploy only approved artifacts
9. record deployment history

Production deployment permissions must be restricted.

---

52. Environment Separation

At minimum:

Development
Staging
Production

Each environment must have separate:

- databases
- credentials
- API keys
- storage
- session secrets
- Tally connections where applicable

Production data must not casually be copied into development.

If production-like data is required for testing, it must be anonymized or otherwise safely transformed.

---

53. Administrator Protection

Administrator accounts are high-value targets.

Admin accounts should use:

- strong authentication
- MFA where possible
- individually identifiable accounts
- restricted privileged permissions
- session monitoring
- audit logging

Do not use a shared administrator account for routine work.

---

54. Break-Glass Access

A controlled emergency access mechanism may be introduced for severe incidents.

If implemented, it must:

- be rare
- require strong authentication
- be heavily audited
- be time-limited where practical
- be reviewed after use
- not become normal operating access

TBD — Security/Operations Decision Required

---

55. Security Headers

Configure appropriate web security headers, including where applicable:

Content-Security-Policy
Strict-Transport-Security
X-Content-Type-Options
Referrer-Policy
Permissions-Policy
Frame protection

The final CSP must be tested against the actual frontend and required integrations.

---

56. Browser Security

The frontend must:

- never contain server secrets
- use secure authentication handling
- avoid unsafe HTML injection
- avoid unnecessary sensitive local storage
- use HTTPS
- clear sensitive state on logout
- protect privileged screens through backend authorization

Frontend restrictions are usability controls, not security boundaries.

---

57. Unknown / Mobile Device Access

If browser/mobile access is supported, assume devices may be:

- shared
- lost
- stolen
- infected
- connected through unsafe networks

Therefore:

- sessions must expire
- sensitive operations require authorization
- logout/revocation must be available
- device trust must not replace user authentication
- privileged actions should receive stronger protection

Full device-management features are:

TBD — Scope Decision Required

---

58. Security Testing

Security testing must include application, business, and infrastructure layers.

58.1 Application Security

Test:

- authentication
- authorization
- object-level authorization
- organization/branch isolation
- input validation
- injection resistance
- XSS
- CSRF where applicable
- CORS
- file uploads
- session handling
- rate limiting
- error leakage

58.2 Business Security

Test:

- duplicate invoice posting
- unauthorized discounts
- unauthorized price override
- unauthorized stock adjustment
- unauthorized branch access
- payment manipulation

- payment over-allocation
- credit-limit bypass
- serial double-sale
- batch manipulation
- cancellation abuse
- return abuse
- Tally duplicate synchronization

58.3 Infrastructure Security

Test/review:

- dependency scanning
- container scanning
- TLS configuration
- firewall/network rules
- secret exposure
- deployment permissions
- backup access
- vulnerability scanning
- production configuration

Penetration testing should be considered before production launch.

---

59. Security Incident Response

The incident lifecycle should be:

Detect
↓
Contain
↓
Investigate
↓
Eradicate
↓
Recover
↓
Review

Potential incidents:

- stolen credentials
- unauthorized account

- database breach
- malware
- leaked API key
- Tally credential compromise
- suspicious financial activity
- destructive database change
- compromised deployment pipeline

Incident contacts and escalation procedures:

TBD — Operations Decision Required

---

60. Data Retention and Deletion

Business records must not be deleted merely to remove them from the UI.

Retention must consider:

- accounting requirements
- tax requirements
- audit requirements
- legal requirements
- operational requirements
- security investigation requirements

Posted inventory and financial history should normally be preserved.

Exact retention periods:

TBD — Business/Legal Confirmation Required

---

61. Security vs Convenience

The project deliberately prefers:

Controlled correction
over
silent modification
Audit trail
over
untraceable convenience
Server-side validation

over
frontend assumptions
Recoverability
over
irreversible deletion
Fail closed
over
unsafe success

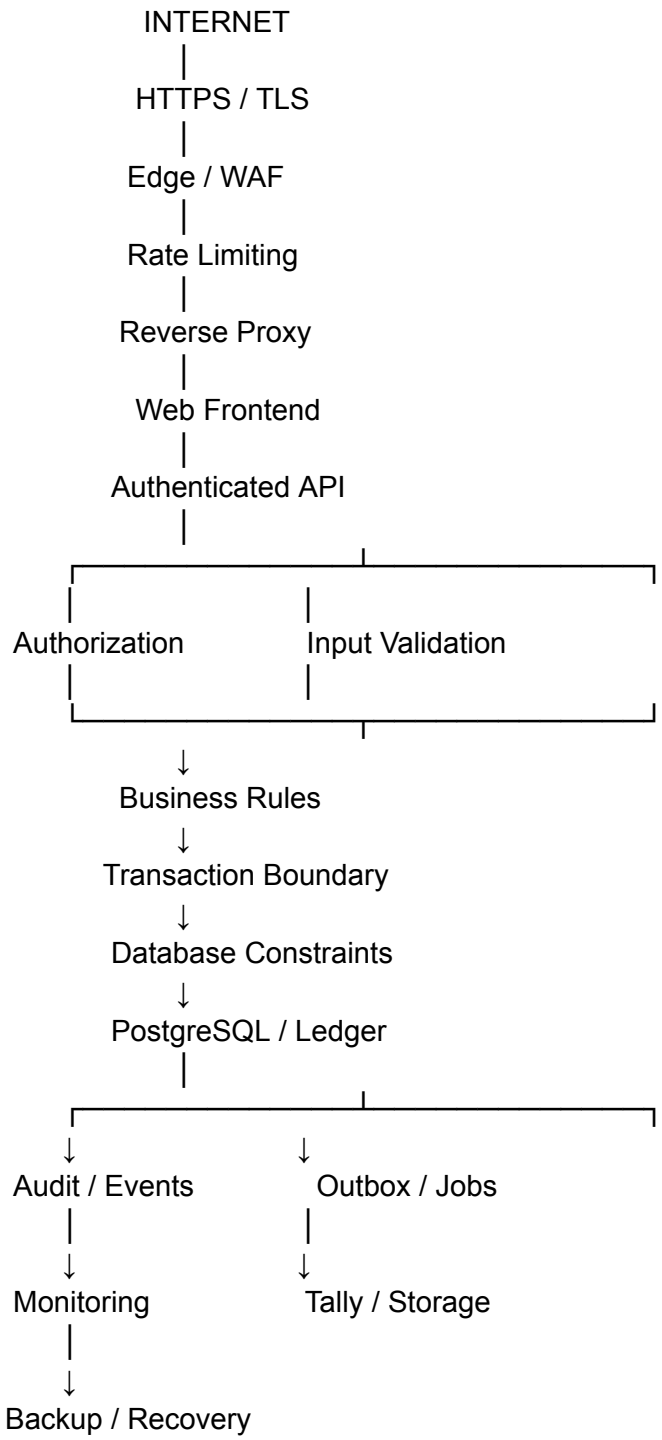
---

62. High-Risk Operations Matrix

Operation	Risk	Required Protection
Login	High	Secure authentication + rate limiting
Password reset	High	Single-use expiring mechanism
User creation	High	Privileged permission + audit
Permission change	Critical	Privileged permission + audit
Price override	High	Server validation + permission + audit
Invoice posting	Critical	Authorization + transaction + audit
Invoice cancellation	Critical	Permission + controlled workflow + audit
Payment posting	Critical	Authorization + transaction + audit
Payment reversal	Critical	Privileged workflow + audit
Stock adjustment	Critical	Permission + reason + transaction + audit
Stock transfer	High	Authorization + transaction + concurrency control
Return posting	High	Validation + transaction + audit
Credit-limit override	Critical	Approval/permission + audit
Tally mapping	Critical	Privileged access + audit
Tally sync	High	Secure integration + idempotency + reconciliation
File upload	High	Type/size/signature controls + malware controls
Sensitive export	High	Authorization + scope + protected delivery
Backup access	Critical	Restricted infrastructure access

---

63. Security Architecture Summary



---

64. Minimum Production Security Checklist

Before production:

- HTTPS configured
- Database not publicly accessible
- Firewall/network rules configured
- Production secrets removed from repository
- Secure authentication implemented
- Password hashing implemented
- Session expiration/revocation implemented
- Authorization enforced server-side
- Organization/branch/location isolation tested
- Object-level authorization tested
- Rate limiting enabled
- Request-size limits enabled
- File upload restrictions enabled
- Security headers configured
- CORS reviewed
- Audit logging enabled
- Sensitive logging reviewed
- Database constraints implemented
- Transaction boundaries tested
- Stock concurrency protection tested
- Serial double-sale protection tested
- Payment allocation protection tested
- Financial transaction protection tested
- Tally credentials protected
- Tally idempotency/reconciliation tested
- Dependency scanning enabled
- Container scanning enabled where applicable
- Production deployment access restricted
- Backups automated
- Backup restoration tested
- Incident response procedure documented
- Security testing completed
- Production security review completed

---

65. Open Security Decisions

The following must be finalized before production:

1. Authentication mechanism
2. Session architecture
3. MFA scope and recovery
4. Password policy
5. Login lockout/rate-limit policy

6. Role/permission structure
7. Organization/branch/warehouse access model
8. Approval thresholds
9. Credit-limit override policy
10. Negative-stock policy
11. Document cancellation policy
12. Audit retention
13. Backup retention
14. RPO/RTO
15. File malware-scanning solution
16. WAF/edge provider
17. Secrets-management solution
18. Monitoring/alerting solution
19. Tally authentication/communication method
20. Production penetration-testing scope
21. Security incident contacts
22. Break-glass access policy
23. Device/session management scope
24. Data retention/deletion periods

All decisions should be recorded explicitly. Unknown policy must not become arbitrary implementation behavior.

---

66. Security Development Rules

Every developer must follow these rules:

1. Never trust client-supplied permissions.
2. Never trust client-calculated totals.
3. Never directly update inventory balances from UI code.
4. Never modify posted financial history directly.
5. Never commit secrets.
6. Never expose database credentials.
7. Never expose Tally credentials.
8. Never log passwords or tokens.
9. Never assume a client-supplied ID is authorized.
10. Never bypass branch/location authorization.
11. Never disable security checks merely to make development easier without documenting the exception.
12. Every security exception must have an owner and removal plan.
13. Security-sensitive changes require review.
14. New dependencies require security/maintenance consideration.
15. Production data must not be copied into development casually.
16. Security fixes must be regression-tested.

---

67. Definition of Protection Done

The ERP protection layer is not production-ready until:

- outer network protection is configured
- HTTPS/TLS is correctly configured
- authentication is secure
- authorization is enforced server-side
- organization/branch/location isolation is tested
- object-level authorization is tested
- critical financial operations are protected
- critical inventory operations are protected
- serial/batch integrity is protected
- database integrity controls exist
- transactions are atomic where required
- audit logging works
- secrets are protected
- file uploads are controlled
- Tally integration is isolated and protected
- backups exist
- restoration has been tested
- security monitoring exists
- dependency/security scanning exists
- incident response is documented
- critical security tests pass
- production security review is completed

---

68. Non-Negotiable Security Principle

Security must exist in the system even when the user interface is bypassed.

A user with a modified browser, custom script, API client, or stolen session must not be able to bypass:

permissions
branch/location restrictions
stock rules
financial rules
document status
audit requirements

The system must assume that anything outside the trusted backend/database boundary can be manipulated.

---

69. Next Security Artifacts

Recommended follow-on documents:

1. `SECURITY-THREAT-MODEL.md`
2. `AUTH.md`
3. `ROLES-PERMISSIONS.md`
4. `BACKUP-RECOVERY.md`
5. `INCIDENT-RESPONSE.md`
6. `SECURITY-TEST-CASES.md`
7. production hardening checklist

These artifacts should refine this baseline rather than contradict it.

---

70. Cross-Document Security Responsibility

Concern	Primary Document	Protection Relationship
Security requirements	`PROTECTION.md`	Defines security baseline
Functional requirements	`SRS_v2.0.md`	Defines required ERP behavior
Business invariants	`BUSINESS-RULES_v1.0.md`	Defines business protections
Architecture	`ARCHITECTURE_v2.0.md`	Defines enforcement boundaries
Database integrity	`DATABASE_v1.0.md`	Defines structural controls
Technology choices	`TECHSTACK_v1.1.md`	Defines implementation mechanisms
API contract	`API.md`	Must enforce security requirements

Tally boundary	`TALLY-INTEGRATION.md`	Must enforce integration security

If documents conflict, security-critical constraints must not be weakened merely for implementation convenience.

---

71. Status

Version: 1.0
Status: Security & Protection Baseline — Ready for Cross-Document Review

This document establishes the security architecture and production protection requirements. Exact technologies, thresholds, retention periods, authentication details, deployment controls, and operational policies remain subject to the explicitly listed decisions.